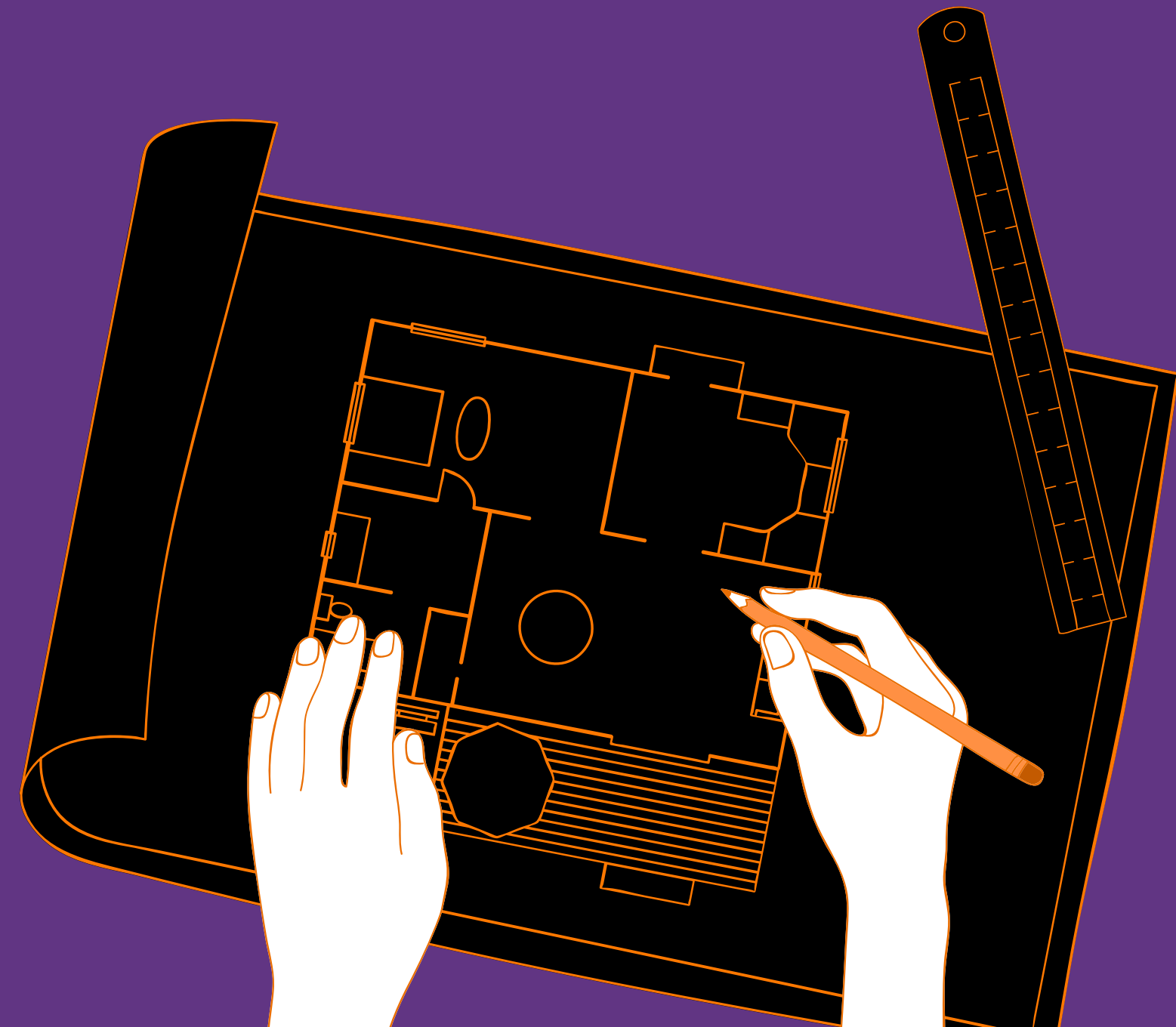




Peachy

Building Reactive Native Apps for Linux



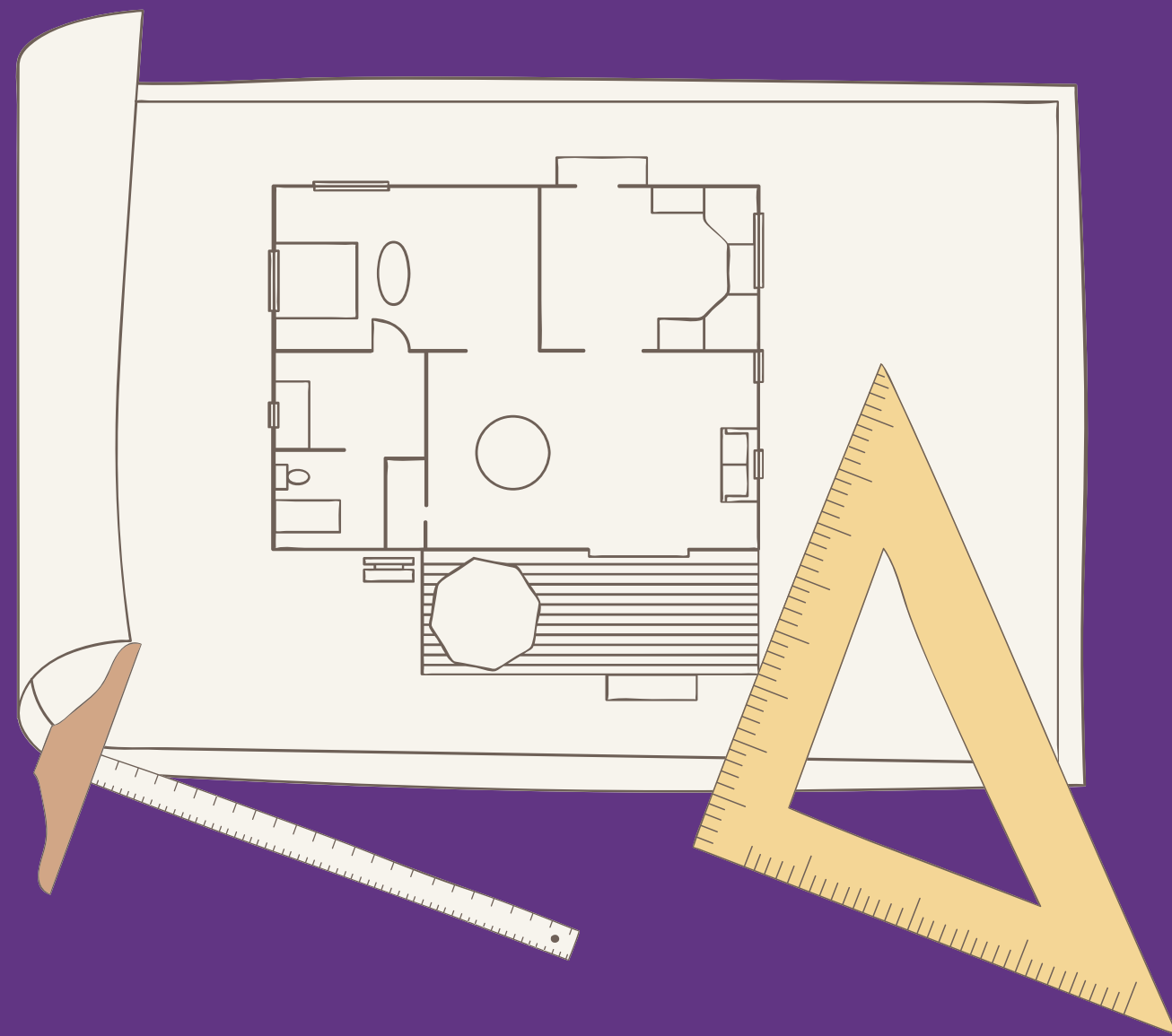
Who am I?

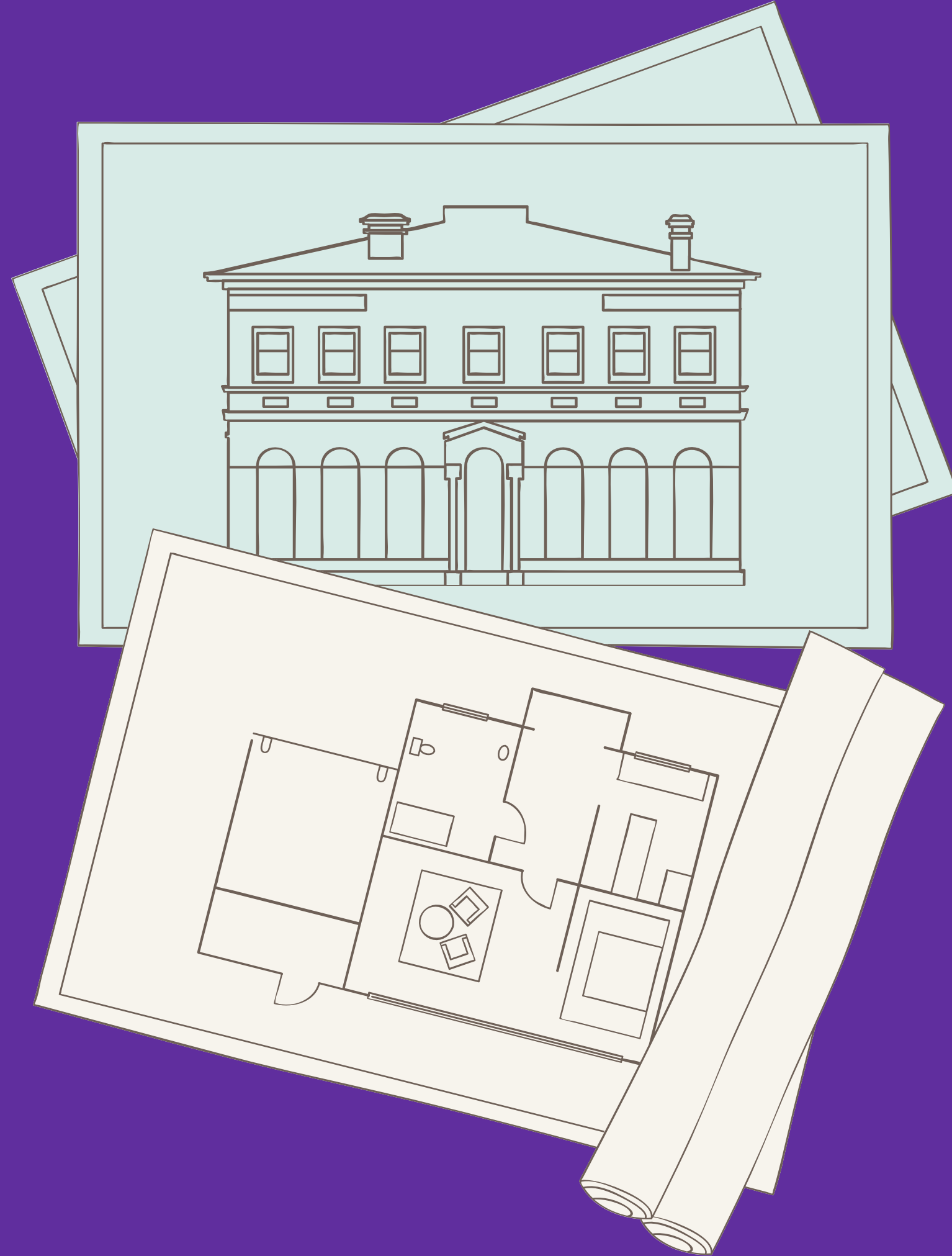
- Angelo Verlain Shema
- vixalien
- University Student
- GSoC 2023/2026
- Maintainer of Decibels & Sticky Notes



What is Peachy?

A framework for writing native GTK4 apps with React, running on GJS, rendering real widgets, not a webview.





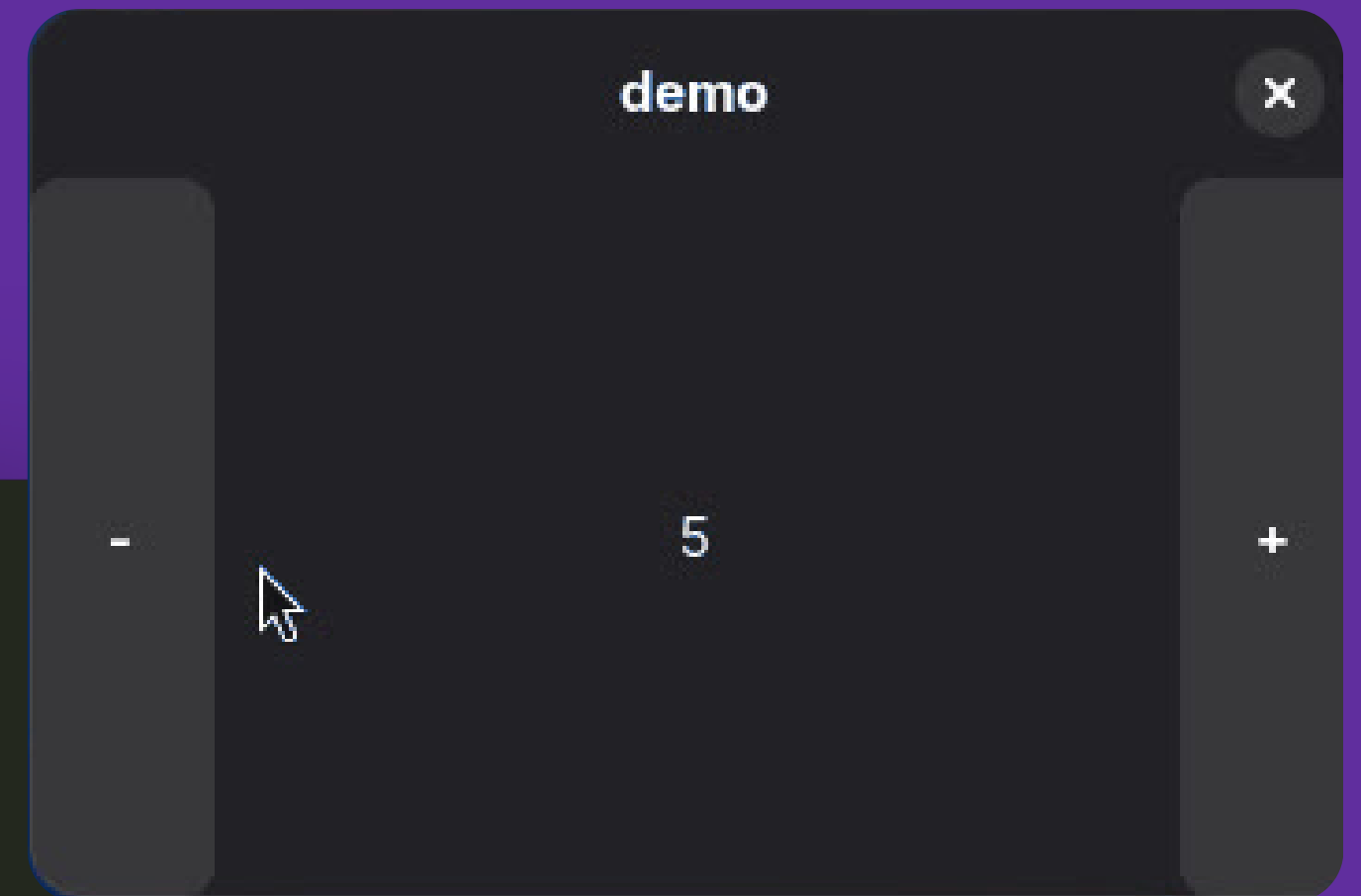
Why Peachy?

- GTK/C/Vala have high friction
- GJS is already proven in GNOME
- Web development produced excellent tooling in the last decade, bundlers, HMR, typed APIs
- Millions of developers already know React

Counter

```
export function Counter() {
  const [count, setCount] = useState(0);

  return (
    <Adw.ToolbarView>
      <Adw.HeaderBar $type="top" />
      <Gtk.Box valign={Gtk.Align.CENTER} marginStart={12} marginEnd={12}>
        <Gtk.Button onClicked={() => setCount(count - 1)}>-</Gtk.Button>
        <Gtk.Label hexpand label={count.toString()} />
        <Gtk.Button onClicked={() => setCount(count + 1)}>+</Gtk.Button>
      </Gtk.Box>
    </Adw.ToolbarView>
  );
}
```



Architecture

- Monorepo layout
- Packages: Core, React, Hooks, Polyfills, Create
- Plugins: React, CSS, Resources, Runner
- Examples

**For Web
developers**

JSX

- JSX is embeddable XML-like syntactic-sugar.
- Transformed into normal JS
- Popularised by React
- Reactive
- No Builder XML

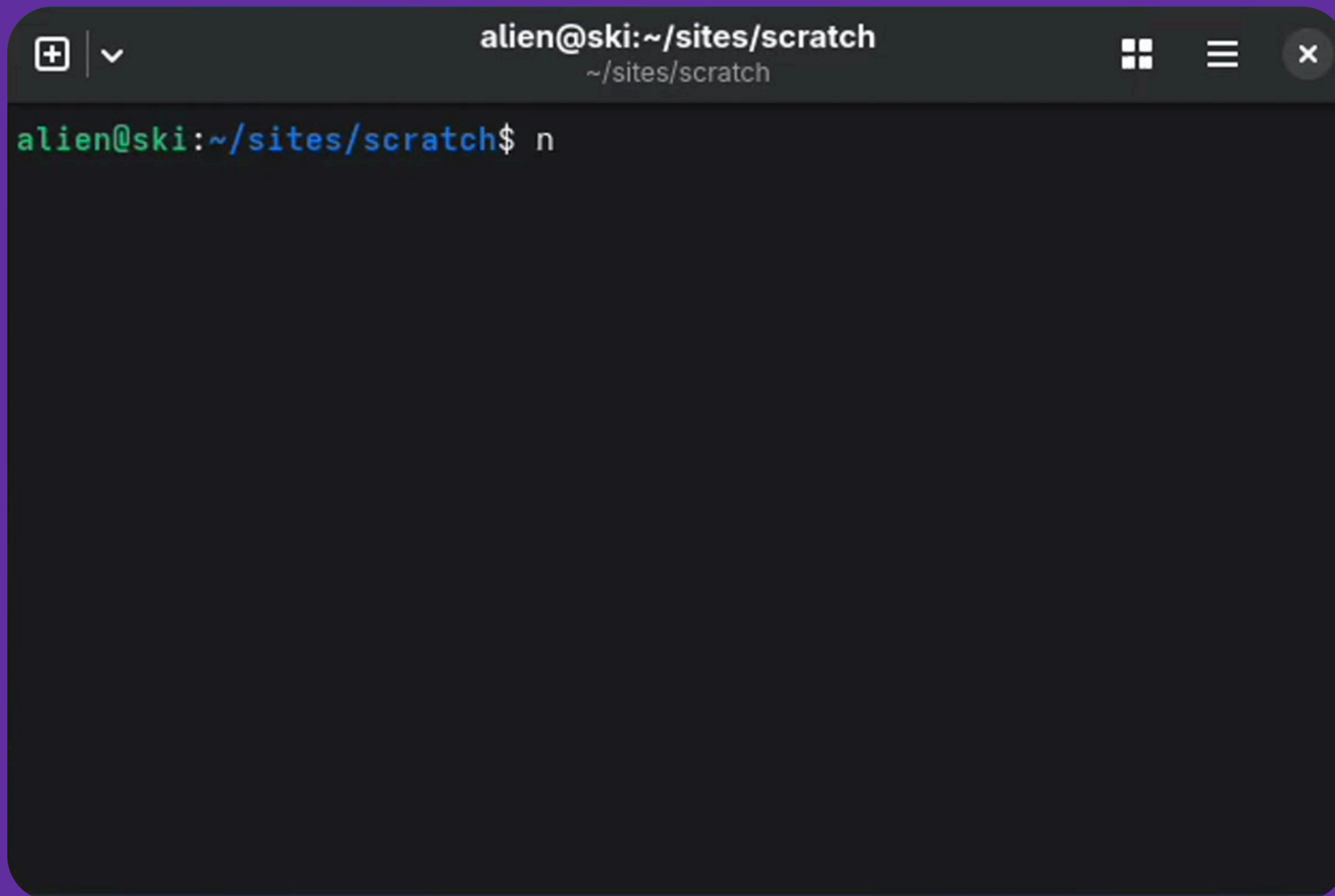
```
import Gtk from "gi://Gtk?version=4.0";
import Adw from "gi://Adw?version=1";

export function Component() {
  const name = "Jane";

  const users = ["John", "Jack", "Jill"]

  return (
    <Adw.ToolbarView>
      <Adw.HeaderBar $type="top" />
      <Gtk.Box>
        {/* Interpolation */}
        <Gtk.Label label={`Your name is ${name}`} />
        {/* Loops */}
        {users.map((user) => (
          <Gtk.Label key={user} label={user} />
        ))}
        {/* Conditionals */}
        {users.length === 0 && <Gtk.Label label="No
users" />}
        {/* Fragments */}
        <>
          <Gtk.Button label="Log in" />
          <Gtk.Button label="Log out" />
        </>
      </Gtk.Box>
    </Adw.ToolbarView>
  );
}
```


Scaffolding

A terminal window with a dark background. The title bar shows 'alien@ski:~/sites/scratch' and '~/sites/scratch'. The prompt is 'alien@ski:~/sites/scratch\$' and the command 'n' is entered.

```
alien@ski:~/sites/scratch$ n
```

```
>_ $ npm create peachy@latest
```

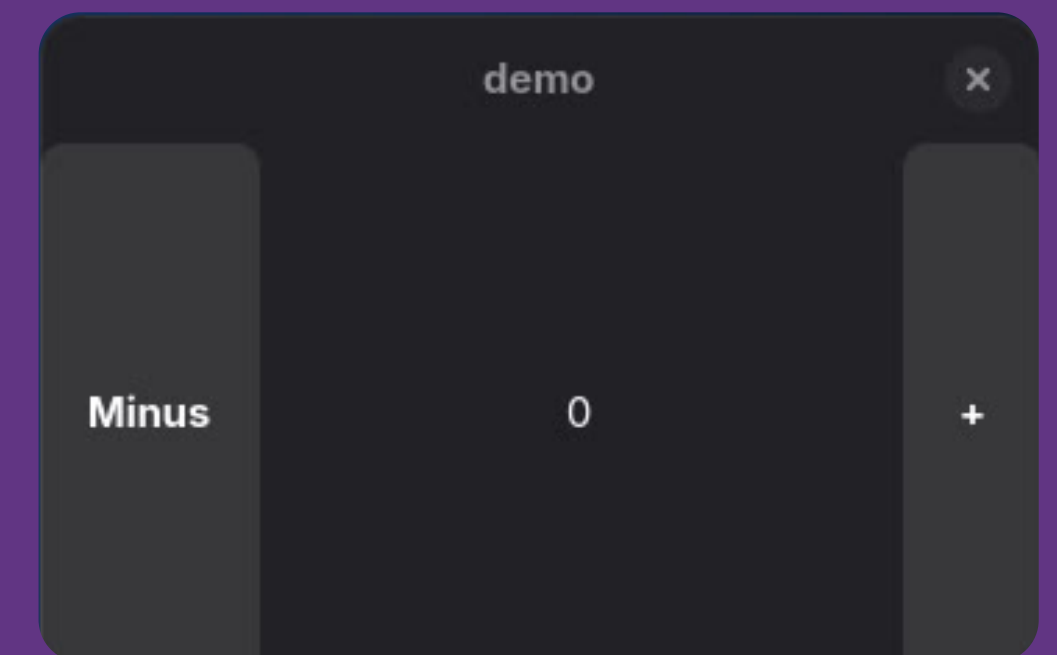
- Get started in 30 seconds
- Requires only Node.js and GJS installed
- Follow the instructions

HMR

The UI updates as soon as you save your code*

```
demo
counter.tsx
src/counter.tsx › function Counter()
12 import Adw from "gi://Adw?version=1";
11 import Gtk from "gi://Gtk?version=4.0";
10
9 import { useState } from "react";
8
7 export function Counter() {
6   const [count, setCount] = useState(0);
5
4   return (
3     <Adw.ToolbarView>
2       <Adw.HeaderBar $type="top" />
1       <Gtk.Box>
13     <Gtk.Button onClicked={() => setCount(count - 1)}>Minus</Gtk.Button>
1       <Gtk.Label hexpand label={count.toString()} />
2       <Gtk.Button onClicked={() => setCount(count + 1)}>+</Gtk.Button>
3     </Gtk.Box>
4   </Adw.ToolbarView>
5 );
6 }
7
```

*most of the time, otherwise your app restarts



Styling

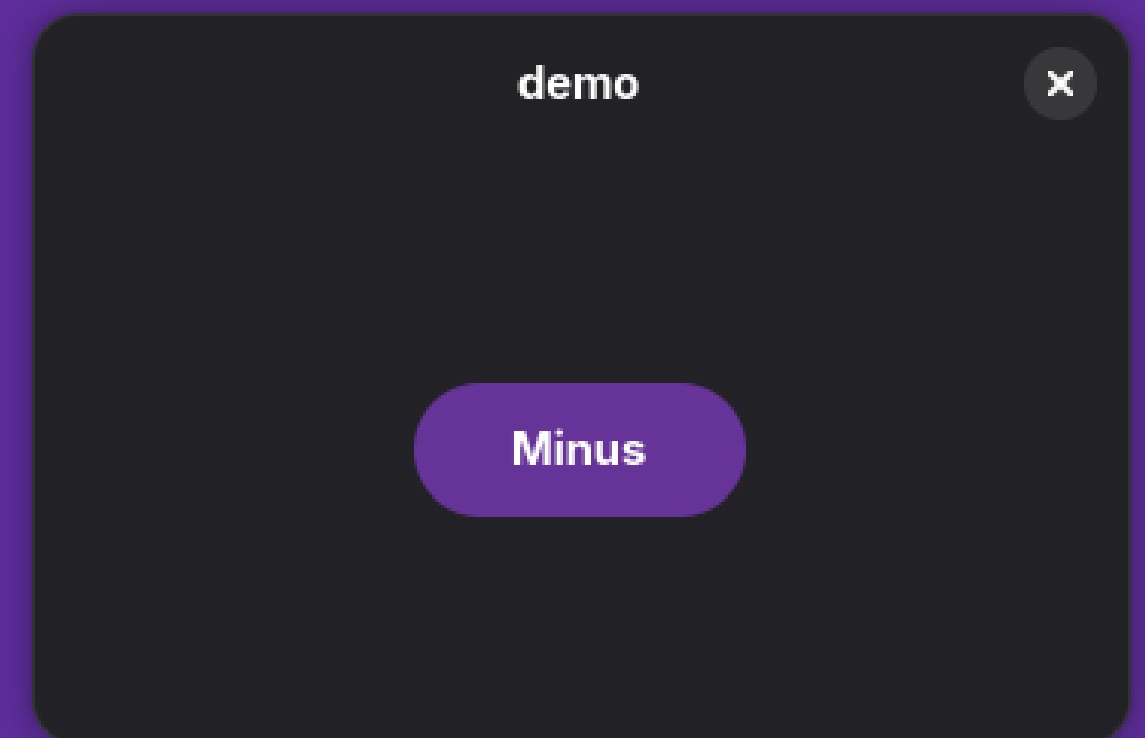
- Support for CSS Modules
- Automatic CSS file configuration

```
// counter.tsx
import Adw from "gi://Adw?version=1";
import Gtk from "gi://Gtk?version=4.0";

import styles from "./counter.module.css";

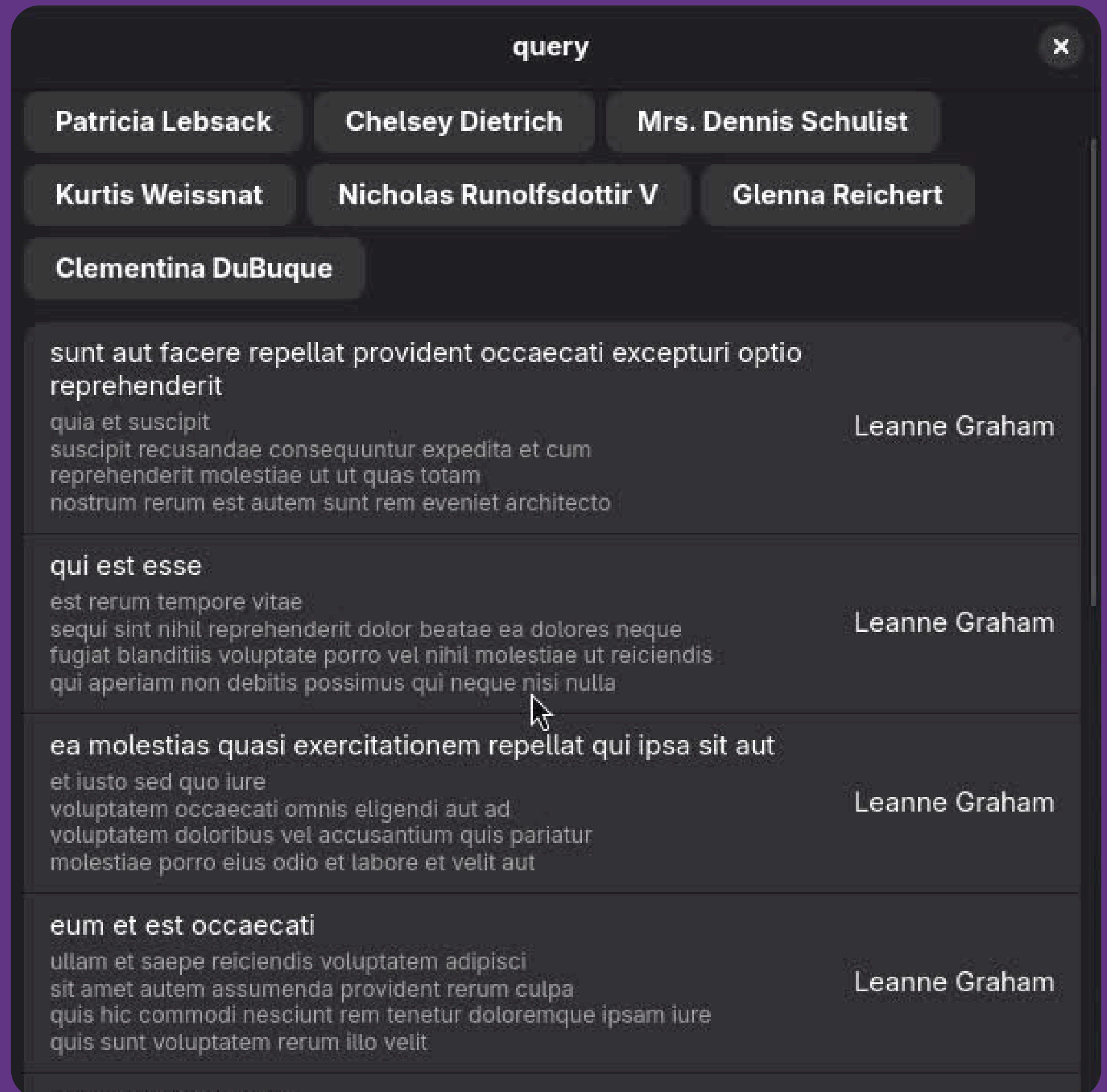
export function Counter() {
  return (
    <Adw.ToolbarView>
      <Adw.HeaderBar $type="top" />
      <Gtk.Box
        halign={Gtk.Align.CENTER}
        valign={Gtk.Align.CENTER}
      >
        <Gtk.Button
          cssClasses={[styles.button, "pill"]}
        >
          Minus
        </Gtk.Button>
      </Gtk.Box>
    </Adw.ToolbarView>
  );
}
```

```
/* counter.module.css */
.button {
  background-color: rebeccapurple;
}
```



Libraries

- Use convenient libraries from npm you already know and love
- Examples: fakerjs, nanostores, swr, react-query, zustand, redux, etc...

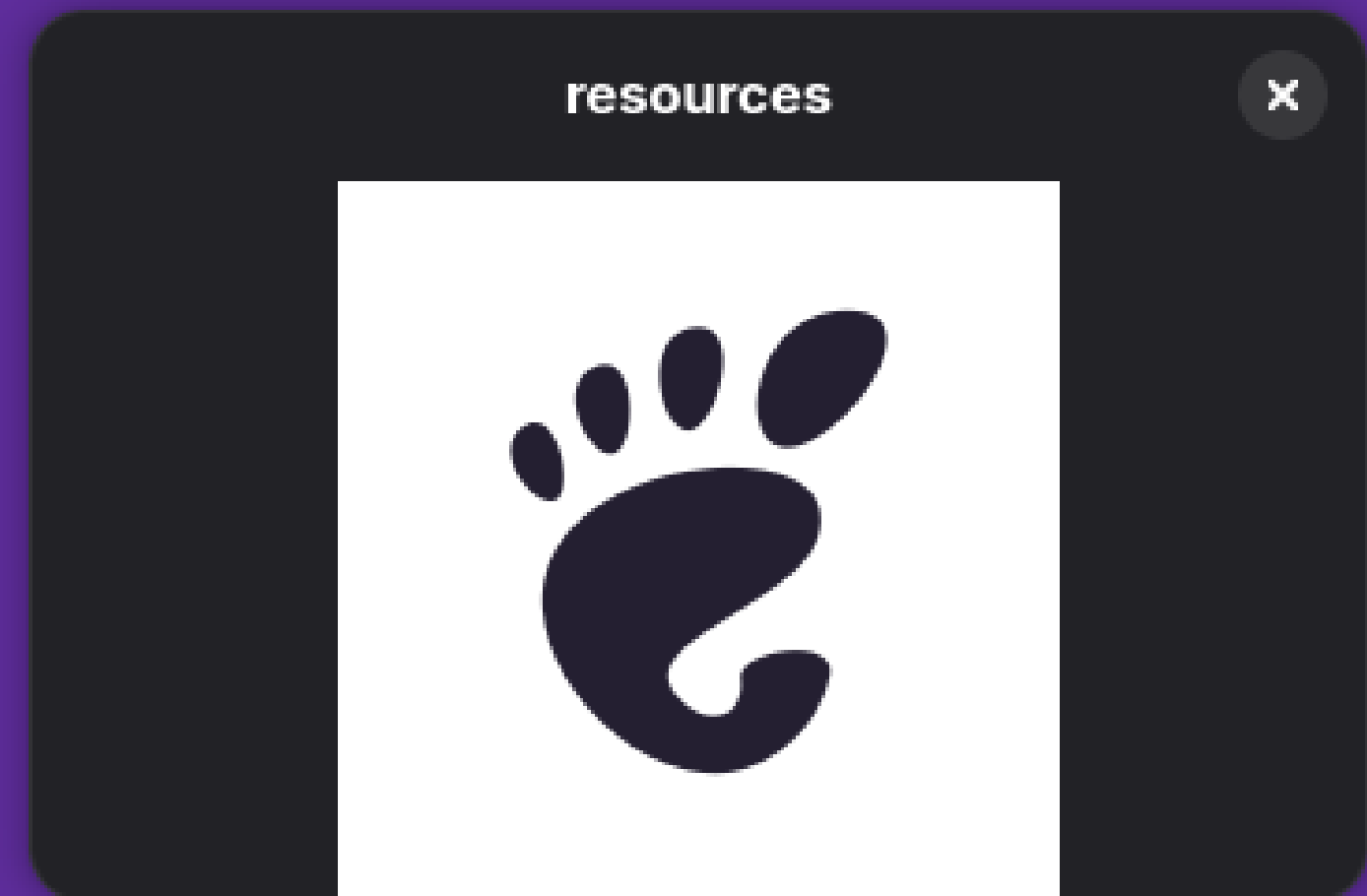


Asset Bundling

```
import Adw from "gi://Adw?version=1";
import Gtk from "gi://Gtk?version=4.0";

import GnomeBrandmark from "../data/
gnome_brandmark.svg";

export function Counter() {
  return (
    <Adw.ToolbarView>
      <Adw.HeaderBar $type="top" />
      <Gtk.Picture
        widthRequest={64}
        heightRequest={64}
        paintable={new Gtk.Svg({ resource:
GnomeBrandmark })}
      />
    </Adw.ToolbarView>
  );
}
```



- Import SVGs, Icons, Images and more...
- They are automatically put into a resource file
- Look Ma! No XML

**For GTK
Developers**

Types!

- Type-safe access to GTK and GNOME APIs
- Type-checking with TypeScript

```
alien@ski:~/sites/peachy-examples/video$ pnpm tsc --noEmit
src/video.tsx:31:12 - error TS2551: Property 'Pictures' does not exist on type '$Exports'. Did you mean 'Picture'?

31      <Gtk.Pictures
      ~~~~~

../node_modules/.pnpm/@peachy+types@2026.7.13/node_modules/@peachy/
types/types/Gtk-4.0.d.ts:37117:17 - 'Picture' is declared here.
37117      Picture: PictureClass
      ~~~~~

Found 1 error in src/video.tsx:31

alien@ski:~/sites/peachy-examples/video$
```

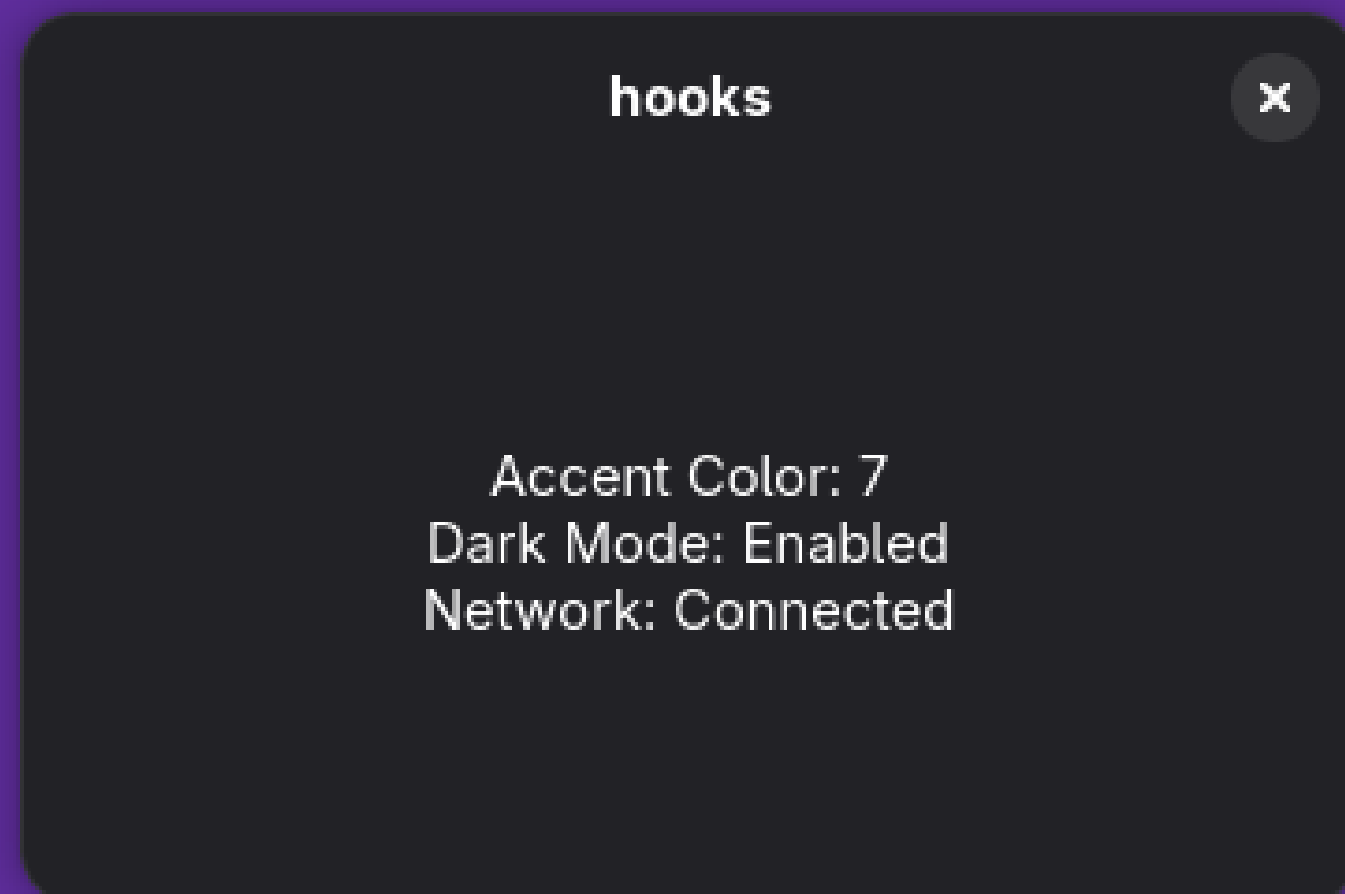
```
demo
counter.tsx counter.module.css
src/counter.tsx > function Counter()
10 import Adw from "gi://Adw?version=1";
9 import Gtk from "gi://Gtk?version=4.0";
8
7 import styles from "./counter.module.css"; 'styles' is decl
6
5 export function Counter() {
4 return (
3 <Adw.ToolbarView>
2 <Adw.HeaderBar style="top" />
center natural width of widget inside the allocation
12 valign={Gtk.Align.C} Property 'Ali' does not exist
CENTER (property) GI.Gtk.AlignEnum["CENTER"]: 3
BASELINE_CENTER (property) GI.Gtk.AlignEnum["BASELINE_CENTER"]
4 <Adw.ToolbarView>
5 );
6 }
```

React

- Use actual React
- Suspense
- Portals
- Concurrent rendering
- Hooks
- React libraries

Hooks

- Convenience hooks to access data
- Includes useAccentColor, useDarkMode, useSetting, useNetworkMonitor, useBinding



```
import Adw from "gi://Adw?version=1";
import Gtk from "gi://Gtk?version=4.0";

import { useAccentColor, useDarkMode } from "@peachy/
hooks";
import { useNetworkMonitor } from "@peachy/hooks/gio";

export function Hooks() {
  const [accentColor] = useAccentColor();
  const [darkMode] = useDarkMode();
  const { connected } = useNetworkMonitor();

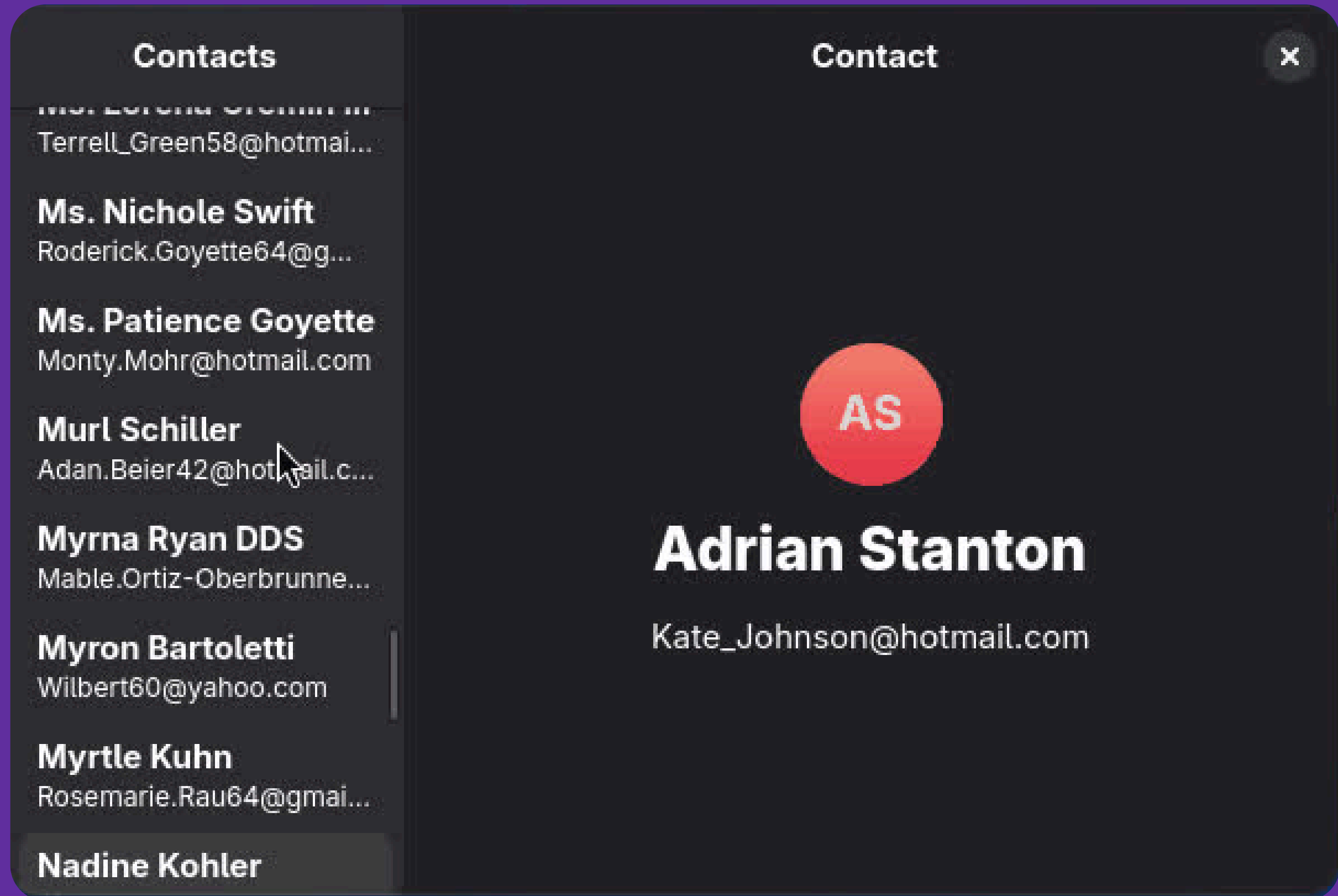
  return (
    <Adw.ToolbarView>
      <Adw.HeaderBar $type="top" />
      <Gtk.Box orientation={Gtk.Orientation.VERTICAL}
valign={Gtk.Align.CENTER}>
        <Gtk.Label label={`Accent Color: ${accentColor}
`} />
        <Gtk.Label label={`Dark Mode: ${darkMode ?
"Enabled" : "Disabled"}`} />
        <Gtk.Label
          label={`Network: ${connected ? "Connected" :
"Disconnected"}`} />
      </Gtk.Box>
    </Adw.ToolbarView>
  );
}
```

GObject Introspection

- **GTK4 and libadwaita:** widgets and adaptive layouts
- **GLib and Gio:** the foundation, settings, files, D-Bus
- **libsoup:** HTTP client and server
- **GStreamer:** media playback and processing
- **libshumate:** map widgets
- **libsecret:** storing passwords and secrets
- **libnotify:** desktop notifications
- **WebKitGTK:** embeddable web views
- **TinySPARQL:** local search and metadata

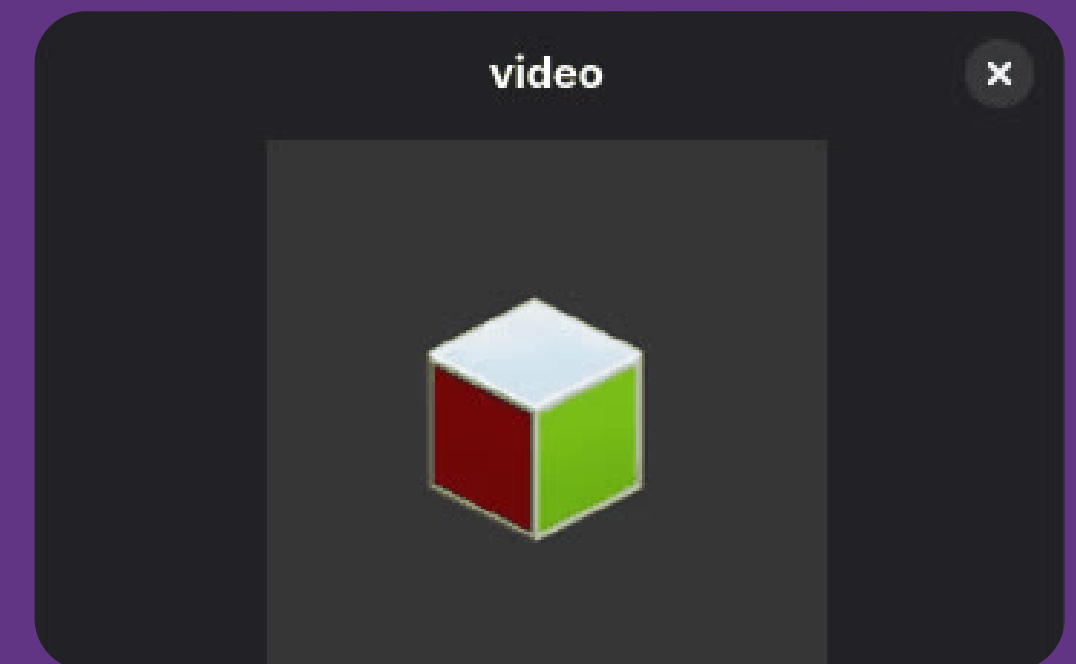
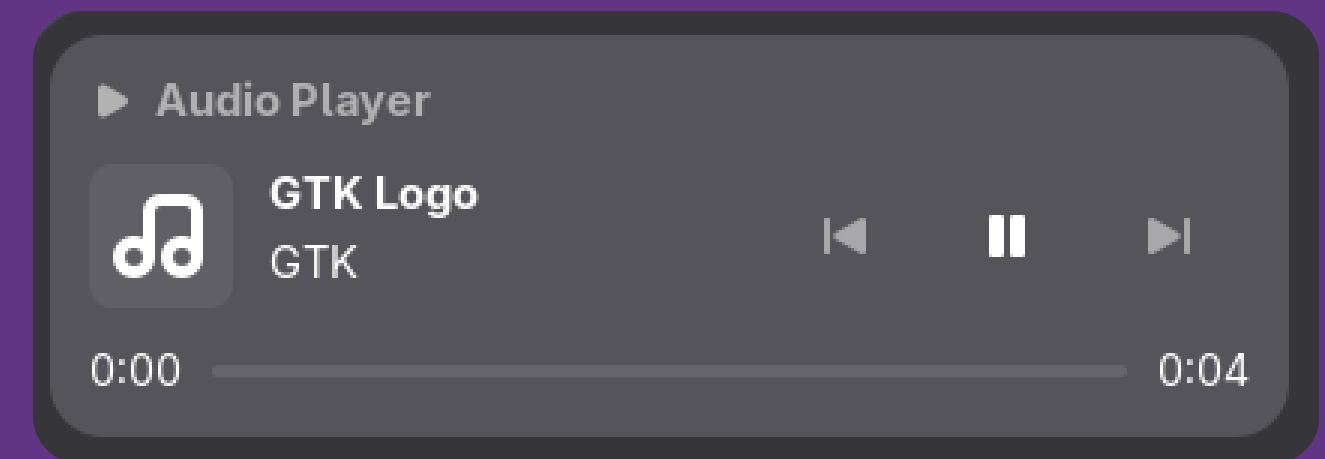
Scale

- Can handle massive datasets well
- Good list management with Gio.ListModel and Gtk.ListView
- Native performance*



Easy D-Bus

Easily communicate with other apps or components with D-Bus (MPRIS example)



```
import { Service, iface, property, method } from "gnim/dbus";

@iface("org.mpris.MediaPlayer2.Player")
export class MprisPlayer extends Service {
  constructor(private mediaFile: Gtk.MediaFile) {
    super();
  }

  @property("s")
  get PlaybackStatus(): string {
    return this.mediaFile.playing ? "Playing" :
    "Paused";
  }

  @method()
  Play(): void {
    this.mediaFile.play();
  }
  // ...
}
```

**For
contributors**

Ecosystem

- girgen: For type generation
- gnim: convenience library for GJS (useSetting, GObject decorators, D-Bus, etc...)
- NPM, Node.js for distribution
- Rolldown, TSDown, TSC
- oxlint, oxfmt
- Changesets

Roadmap

- gettext translation support
- Multi-platform builds
(Windows & macOS)
- Automatic flatpak packaging
- Support for other UI
frameworks (Svelte, Solid,
Vue, etc...)
- Your idea

Thank you

examples: <https://gitlab.gnome.org/vixalien/peachy-examples>

source: <https://gitlab.gnome.org/vixalien/peachy>

matrix: [#typescript:gnome.org](#)

